Collection of all Exercises from Labs A01 and A02

Abstract

This collection of exercises should be implemented by you by the first Checkpoint. Then, you will be asked some questions on a random sample of those exercises comprising of 2S, 2M and 1L exercises. Keep comments on your code at a bare minimum.

LabA01

for Week A01-A02

S1 Function divide for Int numbers

Task: Write a **divide** function for **Int**, in a .hs file.

1 Declare the function type which has two arguments and one return

2 Write the function body. Use the **div** function.

Caution: it is not correct if you directly write

```
a div b
```

as **div** is a function instead of an operator, such as /, ry to think it well.

3 Test your function in Ghci

- Load the file in Ghci using **:load filename**
- Use **:type** or **:t** to show the type of your function
- Call your function with two integer numbers
- What is going to happen if you divide a number by zero ?

```
ghci> :load S1_divide.hs
[1 of 2] Compiling Main             ( S1_divide.hs, interpreted )
Ok, one module loaded.
ghci> :type divide
divide :: Int -> Int -> Int
ghci> divide 8 4
2
```

```
ghci> divide 8 0
*** Exception: divide by zero
```

S2 Calculate the volume of a cylinder

Task: Write a function to calculates the volume of a cylinder given the radius and height in a .hs file

Equation:

$$V = \pi r^2 h$$

Haskells knows pi as a constant.

1 Declare the function type with a return type of **Float** or type class **Floating**

2 Write the function body

3 Test your function in GHCi or using the main function

```
ghci> :load S2_volumeCylinder.hs
[1 of 2] Compiling Main             ( S2_volCylinder.hs, interpreted )
Ok, one module loaded.
ghci> volumeCylinder 2 5
62.831856
```

S3 minOfTwo : find the minimum of two orderable elements

Follow the **max2, max3, and maxOfTwo** function examples in the lecture slides and this lab tutorial

1 Declare the function type using type class **Ord** to ensure the two elements can be compared.

2 Write the function body using **if-then-else** or **Guards**

3 Test your function in **GHCi** or using the **main** function

The following is the example using **GHCi**

```
ghci> :load S3_minOfTwo.hs
[1 of 2] Compiling Main             ( minOfTwo.hs, interpreted )
Ok, one module loaded.
ghci> minOfTwo 2 3
2
ghci> minOfTwo 2.0 3.0
2.0
ghci> minOfTwo 'a' 'b'
'a'
ghci> minOfTwo "London" "Derby"
"Derby"
```

S4 tellSignNum: tell if a number is positive, negative, or zero

Write a funciton to tell if a number is postive, negative, or zero using Guards

1. Function declaration, using two type classes **Ord** and **Num** for the number and the return type is **String. Ord** is required for comparing with 0, while **Num** is required for 0**.**
2. Function body: using Guards to implement comparison with 0 and return "Positive", "Zero", "Negative"

Testing Test using ghci or the main function

```
ghci> :load S4_tellSignNum.hs
[1 of 2] Compiling Main             ( tellSignNum.hs, interpreted )
Ok, one module loaded.
ghci> tellSignNum 5
"Positive"
ghci> tellSignNum (-5)
"Negative"
ghci> tellSignNum 0
"Zero"
```

When you are testing a negative number, you need to use braces to include it or use the $ sign

```
-- Correct
tellSign (-5)
tellSign $ -5

-- Incorrect (Causes a type or parse error)
tellSign -5
```

S5 The product of all numbers in a list: `myProduct`

Task: Define a custom function (e.g., `myProduct`) that computes the product of a list of **numbers** recursively in a .hs file

1. Function declaration: you **MUST** use type class **Num** in function declaration
2. Function body

myProduct should return 1 for an empty list

Hint: Follow the same structural pattern as the `sum_list` function in section 6 , keeping in mind the base case for multiplication.

Testing Test your function in GHCi or using the main function.

The following is the GHCi example:

```
ghci> :load S5_myProduct.hs
[1 of 2] Compiling Main  ( myProduct.hs, interpreted )
Ok, one module loaded.
ghci> myProduct [1..5]
120
ghci> myProduct [1.0..5.0]
120.0
ghci> myProduct [2, 3, 4]
24
```

M1 Filter (keep) all the odd numbers in a list

a isOdd

Write function **isOdd** function which checks if a number is odd in a .hs file.

a.1 Declare the function type Takes an integer (type class **Integral**), and returns a **Bool**.

a.2 Write the function body. Use function **mod** or **rem** to check the remainder of the given number divided by 2 is not 0

the not euqual operator in Haskell is /=

Set the function value to **False** if the number can be divided by 2 (Use mod), otherwise **True**.

b Write a main function (using the do-notation) **b.1** Write hard-coded test code of **isOdd** in the main function

c Create a filter_odd function **c.1** function declararation of **filter_odd**

using type class **Integral**, the input and output are both a list of **Integral**.

c.2 function body

implement filter_odd using **filter** and **isOdd**

The **filter** function can be used to select elements in a list based on a bool predicate function, for example, choosing all even numbers from a list

```
filter predicate_func a_list
```

c.3 Write test code of **filter_odd** in the main function (using the **do**-notation)

d Use the lambda function Follow the lambda function examles in the lecture slides, replace isOdd with a lambda function, write code to output the test results in the main function.

e Compile the programe using ghc e.1 Run the program to ouput text test results for isOdd, filter_odd and filtering odd number using a lambda function.

```
ghc .\M1_filterOdd.hs
[1 of 2] Compiling Main             ( M1_filterOdd.hs, M1_filterOdd.o )
[2 of 2] Linking M1_filterOdd.exe
> .\M1_filterOdd.exe
isOdd 2 : False
filter_odd [1..10]
[1,3,5,7,9]
filter odd numbers in [1..10] using lambda function
[1,3,5,7,9]
```

M2 Free-fall distance sequence calculator

Task: Write a function **freeFallDistSeq** to return the free falling distance at different time (in seconds, such as [1, 2, 3, 4, 5, 6, 7, 8, 9, 10] seconds)

a function freeFallDist, returns the free-fall distane at a given time freeFallDist calculates the distance (in meters) an object falls due to gravity after a given time

$$t$$

(in seconds).

1. Function declaration

Input: a list of time in seconds (Float)

Ouput: the free-fall distance (Float)

2. Function definition

Calculation Formula

$$distance = 0.5 \times g \times t^2$$

where

$$g$$

is the acceleration due to gravity and we take

$$g = 9.8 m/s^2$$

Requirement Use the **let ... in** expression or **where** clause for

$$g$$

b function freeFallDistSeq returns a list of free fall distance call **map** and **freeFallDist** to convert a list of time in seconds to free-fall distances at different times.

1. Function declaration

Input : a list of Float or Floating

Output: a list of Float or Floating

2. Function definiton

Learn from the map function use case when introducing "Functions as First-class Citizens" in the lecture slides, or learn from online resources

map takes a function and a list, and creates a transformed list using the function on each element of the list

```
map func_name a_list
```

here, the source list is the argument of your **freeFallDistSeq** function and the function is **freeFallDist** you have defined.

Testing Use GHCi or writing a main function to test and print the results

Test Case 1

ghci

```
ghci> :load M2_freeFallDistSeq.hs
Ok, one module loaded.
ghci> :browse
freeFallDist :: Float -> Float
freeFallDistSeq :: [Float] -> [Float]
main :: IO ()
ghci> feeFallDistSeq [1..10]
```

main

```
main :: IO()
main = do
    print $ freeFallDistSeq [1..10]
```

Output:

```
[4.9,19.6,44.100002,78.4,122.5,176.40001,240.09999,313.6,396.90002,490.0]
```

Lab A02

S1. 2D Vector Magnitude

Define a function `magnitude :: (Float, Float) -> Float` that takes a 2D coordinate tuple and computes its distance from the origin using the formula

$$\sqrt{x^2 + y^2}$$

.

Hints use **sqrt** for square root.

Test cases

```
ghci> magnitude (3.0, 4.0)
5.0
ghci> magnitude (cos(pi/6), sin(pi/6))
1.0
```

S2. Vector Dot Product Define a vector dot product function **vecdot** which takes two Lists of the **Num** type class and return the dot product.

Write the function declaration and body.

Hints Use **zipWith (*)** and **sum** .

Test cases

```
ghci> vecdot [1,2,3] [4,5,6]
32
ghci> vecdot [1.0, 2.0] [-2.0, 1.0]
0.0
```

S3 Take the second element from a pair tuple

Write a function **select_second** to take the second element from a pair tuple (with arbitrary element types) using pattern matching.

You cannot use the **snd** function.

Hints: Using function argument pattern matching and the wildcard _

Test cases

```
ghci> select_second (1.0, 2.0)
2.0
ghci> select_second (1007001, "Alice")
"Alice"
```

S4 List slicing

Write a **slice** function which takes a start position and end position and extracts the slice from **start** until the **end** (not including the end, assume

$$end \geq start$$

always holds), similar to **lst[start:end]** in Python.

Hints Use **take** and **drop**

Test cases

```
ghci> slice 1 4 [1..5]
[2,3,4]
ghci> slice 3 10 "University of Derby"
"versity"
```

S5 Check a list of Strings if they are Palindromes

Write a function **checkPalindrome** to check a list of Strings, returning the True or False status for each of the String in the list if they are Palindromes.

A List is Palindromes if it is the same as its reverse, such as "madam" and "noon", or "121" and "1331".

Hints Use **map** and **reverse** .

You can write a function to check if a String is palindrome or directly use a lambda function.

Test cases

```
checkPalindrome ["racecar", "level", "noon", "A", ""] -- [True,True,True,True,True]
checkPalindrome [ "racecar", "hello", "121" , "world", "level"] -- [True,False,True,False,T
checkPalindrome ["123", "noon", "haskell", "1331"] -- [False,True,False,True]
```

M1 Extract uppercase characters in a string using two methods

Write an **extract_upper** function to extract all uppercase characters in a string.

Method 1: use **filter**.

Method 2: use **List comprehension**

Hints You can use isUpper from Data.Char by `import Data.Char (isUpper)`

Alternatively, you can check if a character is in ['A'..'Z'] or compare the character directly with 'A' and 'Z'

Test cases M2

M2 Get the columns of a Matrix

Matrix Transpose in Haskell

Background In many areas of computing such as **computer graphics, data science, machine learning, and scientific computing**, data is often represented using **matrices**.

A **matrix** is a rectangular grid of values arranged in **rows and columns**.

An example 2x3 mathematical matrix:

Matrix representation in Haskell

In Haskell, a common way to represent a matrix is as a **list of lists** in row-major order, where:

- The **outer list** represents the list of rows
- Each **inner list** represents one row
- All rows are assumed to have the **same length**

Example:

```haskell
type Matrix a = [[a]]

m :: Matrix Int
m = [ [1, 2, 3], [4, 5, 6] ]
```

This corresponds to the 2x3 mathematical matrix in row major:

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

What is a matrix transpose? The **transpose** of a matrix is obtained by **swapping rows and columns**.

If we have:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

Then the transpose of A, written as , is:

$$A^T = \begin{bmatrix} 1 & 4 & 2 & 5 & 3 & 6 \end{bmatrix}$$

in Haskell list

```
[ [1,4], [2, 5], [3, 6] ]
```

Intuition:

- The **first row** becomes the **first column**
- The **second row** becomes the **second column**
- And so on

Your Tasks **Topic:** Lists of Lists

You are given matrices represented as **lists of rows**:

An example **Int** matrix:

```
m :: Num a => [[a]]
m = [ [1, 2, 3], [4, 5, 6], [7, 8, 9] ]
```

Note: You need to deal with the empty list cases **[ ]** and **[[ ], . . .]**.

Task 1 — Obtain the first column of a matrix using head and map The first element of a list can obtained using **head**.

If **head** is applied to every list of a matrix, we can obtain the first column of the matrix.

Try to write the function **firstColumn** using **map** and **head**.

```
firstColumn:: Num a => [[a]] -> [a]
```

Task 2 — restColumns — Remove the first column of a matrix using tail and map Similary the rest of the matrix (with the first column removed) can be obtained using **tail**.

Try to write the function **restColumns** using **map** and **tail**.

```haskell
restColumns:: Num a => [[a]] -> [[a]]
```

Test cases

```haskell
ghci> firstColumn [[1,2,3],[4,5,6],[7,8,9]] -- [1,4,7]
firstColumn [] -- []
firstColumn [[]] -- []
firstColumn [[],[3,4],[5,6]] --[]
restColumns [[1,2,3],[4,5,6],[7,8,9]] -- [[2,3],[5,6],[8,9]]
restColumns [] -- []
restColumns [[]] -- []
restColumns [[],[3,4],[5,6]] -- []
```

M3 Type-Safe BMI Calculator

In this exercise, you will build a type-safe Body Mass Index (BMI) calculator using custom enumerations, guards, and tuples in Haskell.

Task Requirements:

1. Define an Enumeration (ADT):

   Create a custom data type named `BMILevel` with the following constructors:

   - `Underweight`
   - `Normal`
   - `Overweight`
   - `Obese`
   - *Hint:* Don't forget to derive `Show` and `Eq` for your data type.

2. Implement the Evaluation Function:

   Write a function named `evaluateBMI` with the following type signature:

   Haskell

   ```haskell
   evaluateBMI :: Float -> Float -> (Float, BMILevel)
   ```

   - The function takes two parameters: weight (in kilograms) and height (in meters).
   - You can assume valid inputs where `height > 0`.

3. Calculation and Categorization:

   - Compute the BMI using the formula:

   $$\frac{\text{weight}}{\text{height}^2}$$

   (hint: use a `where` clause to keep your code clean).

- Use guards to map the calculated BMI to the correct `BMILevel` based on standard thresholds:
  - Less than 18.5 → `Underweight`
  - 18.5 to less than 25.0 → `Normal`
  - 25.0 to less than 30.0 → `Overweight`
  - 30.0 and above → `Obese`

4. Return Value:

   Return a tuple containing the raw calculated `Float` value as the first element and the `BMILevel` enumeration constructor as the second element.

Test cases

```
evaluateBMI 70.0 1.75 -- (22.857143,Normal)
evaluateBMI 85.0 1.75 -- (27.755102,Overweight)
evaluateBMI 100.0 1.7 -- (34.602074,Obese)
evaluateBMI 53.0 1.7 -- (18.3391,Underweight)
```

L1 Credit card number checking

The final task for this lab is to write a programme that can check a credit card number for validity (to protect from e.g. input errors). Credit card number formats are checked using an algorithm called Luhn's algorithm, which creates a simple checksum.

Luhn's algorithm starts at the right and doubles every 2nd number from the least significant digit (the rightmost side); if doubling gives a 2 digit number the digits are added to give a single digit (e.g. 16 would be converted to 7, for numbers ranging between 10 and 18, it is equivalent to subtract 9 from the number). You then sum all values and if the number can be divided by 10 without a remainder you have a valid credit card number.

Here is an example (note that the algorithm doesn't say a Credit Card number needs to have a specific length):

Original number	9	9	2	7	3	9	8	7	1	3
Double every other	18		4		6		16		2	
Remove 2 digits if needed	9						7			
Numbers to be added for final check	9	9	4	7	6	9	7	8	2	3

The sum of the numbers in the last row is 63; 63 cannot be divided by 10 without remainder so the number is invalid.

Task

Create a programme that checks a credit card number.

Function signature

```
checkCCNumber :: String -> Bool
```

It returns **True** if a number has a valid format.

Hints: I am not providing you many details for this but here are some hints:

1. You want to split the problem into several functions, like

convert **String** (a list of digit **Char**s) into a list of **Int**s

dealing with doubling every other number etc.

2. To use character functions, you need to import Data.Char functions, such as

```
chr, ord, isAlpha, isDigit, isUpper, isLower
```

3. You will need to list pattern matching quite a few times, such as **x:xs**, but sometimes you need more detail it could also be seen as **x:y:xs**, which hopefully helps with '**every other**' !
4. To find every other number from the least significant digit (the rightmost side), you may **reverse** the list first.

Test Cases

```
checkCCNumber "378282246310005" -- True
checkCCNumber "422222222222222" -- False
checkCCNumber "4012888888881881" -- True
```